
Thundersoft 2-week Python & GenAI Learning Plan – V3.0

By Kalyani Polepeddy / Narendra Babu Koppera

Motivation

"Success doesn't come from what you know. It comes from what you consistently practice, build, and improve every single day."

Course Objective

This learning plan is designed to build a **strong foundation in Python programming, Automation Framework Development, Android Scripting, and Generative AI** through structured learning and extensive hands-on practice.

The roadmap focuses on:

- Learning Python from fundamentals to advanced concepts.
- Building automation-ready scripting skills.
- Understanding testing frameworks and best coding practices.
- Developing Android automation scripts using ADB and Python.
- Exploring modern Generative AI technologies and building real-world applications.
- Preparing for product-based company interviews through practical problem-solving and projects.

"Learn ➜ Practice ➜ Build ➜ Debug ➜ Improve ➜ Repeat."

Course Roadmap

Part I – Python Programming (2 Weeks)

Week 1 – Core Python Foundations

- Day 1–2: Python Basics
- Day 3–4: Control Structures
- Day 5: Functions & Lambda
- Day 6–7: Data Structures & Comprehensions

Week 2 – Automation-Oriented Python

- Day 8: File Handling
- Day 9: Exception Handling
- Day 10–11: Subprocess
- Day 12: Modules, Packages & Regex
- Day 13: Pytest + OOP
- Day 14: Python Automation Framework
- Day 15: Integration Project

Part II – Generative AI (7 Days)

- Day 16: GenAI & LLM Fundamentals
- Day 17: RAG, Embeddings & Vector Databases
- Day 18: LangChain, LlamaIndex & Ollama
- Day 19: AI Agents & Prompt Engineering
- Day 20: Production GenAI (FastAPI & Docker)
- Day 21: Document Q&A Assistant
- Day 22: End-to-End GenAI Capstone Project

Part III – Android Automation Scripting

- ADB & Device Basics
- Wi-Fi Automation
- Camera Automation
- Bluetooth Automation
- Log Collection & Analysis
- APK Automation
- Memory & Performance
- Battery Automation
- Threading
- Reports
- GNSS (GPS)
- Display

- Telephony / WWAN
- Audio
- Video
- Sensors
- Android Framework
- Storage
- Libraries & Vendor Files

Learning Outcome

By the end of this roadmap, you will be able to:

Write clean, production-ready Python code.

Build reusable automation scripts using Python.

Work confidently with files, JSON, Regex, Exceptions, Modules, OOP, and Pytest.

Develop Android automation scripts using ADB and Subprocess.

Design a basic Python Automation Framework.

Understand and build Retrieval-Augmented Generation (RAG) applications.

Build GenAI applications using LangChain, FAISS, ChromaDB, FastAPI, and OpenAI APIs.

Solve interview coding questions with confidence.

Final Thought

"The expert in anything was once a beginner who refused to give up."

Consistency beats intensity.
Practice every day, write code every day, debug every day—and success will follow.

Part I – Python Programming (2 Weeks)

Week 1 – Core Python Foundations

Day 1–2: Python Syntax & Basics

Topics

- Python Syntax
- Variables
- Data Types
    - int
    - float
    - string
    - boolean
- PEP 8 Standards
- Comments
- Docstrings
- Input/Output Functions
    - `print()`
    - `input()`
- Operators
    - Arithmetic
    - Comparison
    - Logical
- Built-in Functions
    - `len()`
    - `type()`
    - `str()`
    - `int()`
    - `float()`

Practice Programs

1. Print **"Hello QA World"**
2. Take user input for name and greet them.
3. Simple calculator (Addition, Subtraction, Multiplication, Division).
4. Swap two variables.
5. Check the data type of a variable.
6. Convert string to integer and float.
7. Compare two numbers.
8. Find the area of a rectangle.
9. Armstrong Number Checker.
10. Palindrome Checker (String & Integer).

Day 3–4: Control Structures

Topics

- Conditional Statements
    - if
    - elif
    - else
- Loops
    - for
    - while
- Loop Control Statements
    - break
    - continue
    - pass

Practice Programs

1. Even/Odd Checker.
2. Largest of Three Numbers.
3. Multiplication Table.
4. Factorial using Loop.
5. Fibonacci Series (Iteration & Recursion).

6. Reverse a String (3 Ways).
7. Count Vowels in a String.
8. Validate Login using Username & Password.
9. Prime Number Checker.
10. Balanced Brackets Validation (`()`, `{}`, `[]`).

Day 5: Functions & Lambda

Topics

- Defining Functions
- Parameters
- Return Values
- Default Arguments
- Keyword Arguments
- Positional vs Keyword Arguments
- Variable Arguments
    - `*args`
    - `**kwargs`
- Anonymous Functions (`lambda`)
- `map()`
- `filter()`
- `reduce()`
- Generator Functions (`yield`)
- Generator vs Iterator
- Loops vs Generator vs Iterator (Which one is better and why?)

Practice Programs

1. Function to calculate Factorial.
2. Lambda function to calculate Square.
3. Filter Even Numbers from a List.
4. Convert Strings to Uppercase using `map()`.
5. Sum a List using `reduce()`.

6. Password Strength Validator (Regex).
7. Generate Random Test Case IDs.
8. Lambda Function to Check Prime Number.
9. Generate Pascal's Triangle.
10. Anagram Checker.
11. Function with Default Arguments.
12. Function with Keyword Arguments.
13. Function demonstrating Positional vs Keyword Arguments.
14. Fibonacci Generator using `yield`.
15. Function using `*args` and `**kwargs`.

Day 6–7: Data Structures & Methods

Topics

Lists

- append()
- extend()
- insert()
- remove()
- pop()
- sort()
- reverse()

Tuples

- Immutable Nature
- count()
- index()

Sets

- add()
- update()
- union()
- intersection()

- difference()

Dictionaries

- keys()
- values()
- items()
- get()
- update()
- pop()

Comprehensions

- List Comprehension
- Set Comprehension
- Dictionary Comprehension

Practice Programs

1. Store Test Case Names in a List and Sort Them.
2. Store Test IDs in a Tuple and Access by Index.
3. Perform Set Operations for Unique Test Data.
4. Create a Dictionary for Test Case → Status Mapping.
5. Generate Squares using List Comprehension.
6. Map Numbers to Cubes using Dictionary Comprehension.
7. Find Unique Vowels using Set Comprehension.
8. Merge Two Dictionaries.
9. Nested Dictionary with List Values (Append, Modify, Remove).
10. Find Second Largest and Second Minimum Number.
11. List slicing methods all ways
12. what is lambda function and how to use it ?
13. Lambada vs list comprehnsion
14. iterator vs generator vs loop vs list which is more memory efficient
15. what is hash table and hashability?
16. Can we use set inside set or set inside list ? Why ?
17. Why dictionary keys unque ? Why set not allow duplicates
18. how python memory handling work ? What is referrence counting and garbage collector ?

19.What is lazy evalution in python where it is there ?

20.What is deep copy and shallowcopy with example?

21.What is growing list and growing dicitionary how it will work?

22.Sort vs sorted in python?

Week 1 Summary

Concepts Covered

- Python Syntax
- Variables & Data Types
- Input/Output
- Operators
- Built-in Functions
- PEP 8 Standards
- Comments & Docstrings
- Conditional Statements
- Loops
- Functions
- Lambda Functions
- `map()`
- `filter()`
- `reduce()`
- Default & Keyword Arguments
- `*args`, `**kwargs`
- Generators & Iterators
- Lists
- Tuples
- Sets
- Dictionaries
- Comprehensions

Hands-on Practice

- **45+ Python Programs**

- **10+ Logic Building Problems**
- **Strong Foundation for Automation Testing**

Part I – Python Programming (2 Weeks)

Week 2 – Automation-Oriented Python & Framework Development

Day 8: File Handling (Expanded)

Topics

File Handling Basics

- Read and Write Text Files
- CSV File Handling
- JSON File Handling

File Methods

- `read()`
- `readlines()`
- `write()`
- `writelines()`

JSON Handling

- JSON Parsing
- JSON Serialization
- Using `json` module

Automation Usage

- Reading test data
- Writing execution results
- Creating configuration files
- Generating reports

Practice Programs

1. Write test case results to a text file.
2. Read test case names from a file.
3. Append execution logs to a file.
4. Parse CSV test data.
5. Write JSON test report.
6. Read JSON configuration file.
7. Count number of lines in a file.
8. Search for a keyword in a file.
9. Logical: Word frequency counter in a file.
10. Logical: Compare two text files and print mismatched lines.

Additional JSON Handling Programs

11. Read a JSON file containing multiple test cases and print them in tabular format.
12. Update a JSON configuration file with new key-value pairs
    Example:

```
{
 "timeout":30
}
```

13. Merge two JSON files into one consolidated report.
14. Validate JSON structure (check required keys exist).
15. Convert Python dictionary into JSON string and save to file.
16. Load JSON test data and filter cases by status.

Example:

- Passed test cases
- Failed test cases

17. Pretty-print JSON data with indentation for readability.
18. Handle JSON decode errors gracefully using exception handling.
19. Compare two JSON files and highlight differences.
20. Logical: Build a mini JSON-based test case management system.

Day 9: Exception Handling

Topics

Exception Handling Concepts

- try
- except
- finally
- raise

Custom Exceptions

- Creating user-defined exceptions
- Handling application-specific failures

Common Python Exceptions

- ValueError
- KeyError
- IndexError
- FileNotFoundError
- JSONDecodeError

Automation Usage

- Handling script failures
- Reporting errors
- Making automation scripts stable

Practice Programs

1. Handle division by zero.
2. Handle missing file exception.
3. Handle invalid input type.
4. Handle missing dictionary key.
5. Handle index out of range.
6. Handle JSON decode error.
7. Handle subprocess failure.
8. Handle invalid conversion (String → Integer).

9. Logical: Create custom exception for invalid test case.
10. Logical: Password strength validator with exception handling (Regex).

Day 10–11: Subprocess Module

Topics

Subprocess Basics

- `subprocess.run()`
- `subprocess.check_output()`

Automation Usage

- Execute external commands
- Run test scripts
- Capture command output
- Automate test execution

Practice Programs

1. Run echo command:

```
Hello Automation
```

2. Run `dir` or `ls` command to list files.
3. Execute Python script using subprocess.
4. Capture output of date command.
5. Run batch file for test execution.
6. Run pytest suite and log results.
7. Execute multiple commands sequentially.
8. Handle subprocess errors gracefully.
9. Redirect subprocess output to a file.
10. Logical: Automate execution of multiple test scripts and collate results.

Day 12: Modules, Packages & Regex (Expanded)

Topics

Module

A module is a single Python file (`.py`) containing functions, classes, or variables.

Example:

`math.py`

Library

A collection of modules bundled together.

Example:

- Python Standard Library

Package

A directory containing multiple modules with an `__init__.py` file that allows hierarchical organization.

Example:

`requests package`

Import Usage

- `import module_name`
- `from module import function`
- `import package.module`

Built-in Modules to Practice

- os
- sys
- datetime
- subprocess
- math
- random

- re (Regex)

Practice Programs – Modules

1. Import math and calculate:
- Square root
- Power
- Floor
- Ceiling
2. Import random and generate test data:
- Test IDs
- Passwords
3. Import os and list files from directory.
4. Import sys and print:
- Python version
- Script arguments
5. Create custom module for test utilities.

Example:

`utils.py`

6. Import custom module and use functions.
7. Use datetime for timestamps in logs.
8. Use subprocess inside custom module.
9. Logical: Organize files by extension.

Examples:

- PDF
- JPG
- TXT

10. Logical: Build reusable test helper library.

Regex Module (re) Practice Programs

11. Use `re.match()` to validate email IDs.
12. Use `re.search()` to find keywords.

13. Use `re.findall()` to extract numbers.
14. Use `re.sub()` to mask:

- Aadhaar numbers
- Mobile numbers

15. Logical: Validate date format.

Format:

```
DD/MM/YYYY
```

16. Logical: Validate IPv4 addresses.
17. Logical: Extract URLs from text files.
18. Logical: Build regex-based password strength checker.
19. Logical: Implement custom strip() using regex.
20. Logical: Regex-based log analyzer.

Search:

- CRASH
- ANR
- FATAL

Day 13: Pytest Basics & OOP

Pytest Topics

- Writing Unit Tests
- Assertions
- Parametrize Tests
- Fixtures
- Running pytest from command line
- Running pytest using subprocess
- Pytest reports
- HTML Report Generation
- Skip and Mark tests

OOP Concepts

Basics

- Class
- Object
- self
- Constructor

Advanced Concepts

- super()
- Inheritance
- Method Overriding
- Method Overloading
- MRO (Method Resolution Order)
- Encapsulation
- Abstraction
- Abstract Method
- Decorator
- Iterator
- generator

Practice Programs

1. Test addition function.
2. Test palindrome function.
3. Test factorial function.
4. Test leap year function.
5. Test password validator.
6. Run pytest through command line.
7. Run pytest using subprocess.
8. Generate pytest HTML report.
9. Logical: Parametrize test cases for multiple inputs.
10. Logical: Skip/Mark tests based on conditions.
11. OOP: Create TestCase class with attributes and methods.

12.OOP: Demonstrate inheritance using base and child classes.

13.OOP: Demonstrate method overriding.

14.OOP: Demonstrate method overloading using default arguments.

15.OOP: Explore MRO using multiple inheritance.

Day 14: Python Automation Framework Development

Topics

Framework Structure

- Project folder organization
- Reusable utilities
- Configuration management
- Logging mechanism
- Exception handling
- Test execution engine
- Reporting module

Framework Components

Test Data Management

- CSV
- JSON
- Configuration files

Utility Layer

- File utilities
- Logging utilities
- Common helper functions

Execution Layer

- Test runner
- Subprocess execution
- Pytest integration

Reporting Layer

- Pass/Fail reports
- Execution summary
- HTML reports

Practice Project

Build a basic Python Automation Framework:

Features:

1. Read test cases from JSON/CSV.
2. Execute automation scripts.
3. Capture execution logs.
4. Handle failures using exceptions.
5. Generate execution reports.
6. Maintain reusable utility modules.

Day 15: Integration Project

Project: Python-Based Test Automation System

Requirements

Test Data Processing

- Parse test data from CSV/JSON.

Test Execution

- Run automated tests using subprocess.

Logging

- Store execution logs.

Error Handling

- Handle failures gracefully.

Report Generation

Generate summary reports using:

- Lists
- Dictionaries
- Comprehensions

Regex Integration

Implement:

- Validate email IDs.
- Validate mobile numbers.
- Extract test case IDs from logs.
- Search error codes in log files.
- Validate date formats.

Week 2 Summary

Concepts Covered

- File Handling
- CSV Processing
- JSON Parsing
- Exception Handling
- Custom Exceptions
- Subprocess Automation
- Modules
- Packages
- Regex
- Pytest
- OOP Concepts
- Automation Framework Design
- Reporting
- Logging
- Integration Projects

Hands-on Practice Outcome

- **80+ Python Automation Programs**
- **Framework Development Practice**
- **Testing Automation Skills**
- **Interview-Oriented Projects**

Python 2-Week Completion Outcome

After completing Part I, Now You are able to:

Write structured Python programs.

Handle files, JSON, CSV, and logs.

Build reusable Python modules.

Create automation scripts using subprocess.

Write and execute pytest test cases.

Apply OOP concepts in automation.

Develop a basic Python automation framework.

Build real-world automation projects.

Part II – Generative AI (7 Days Roadmap)

Day 16: GenAI & LLM Fundamentals

Topics

1. What is Generative AI?

- Introduction to Generative AI
- Difference between Traditional AI and Generative AI
- Real-world applications of Generative AI

Examples:

- Text Generation
- Image Generation
- Code Generation

- Document Summarization
- Question Answering Systems

---

2. What are LLMs?

- Introduction to Large Language Models (LLMs)
- How LLMs generate human-like responses
- Training data and model capabilities
- Examples of popular LLMs

---

3. How LLMs Work (High Level)

- Data Collection
- Pre-training
- Fine-tuning
- Inference
- Response Generation

---

4. Transformer Basics

- Introduction to Transformer Architecture
- Encoder and Decoder concepts
- Attention Mechanism
- Self-Attention
- Why Transformers are powerful for language tasks

---

5. Tokens & Tokenization

- What are tokens?
- How text is converted into tokens
- Token limits
- Token usage in LLM APIs

---

6. Context Window

- Understanding context length

- How LLMs remember previous information
- Limitations of context windows

---

7. Temperature, Top-p & Top-k

Temperature

- Controls randomness of responses.

Top-p

- Controls probability-based token selection.

Top-k

- Selects from top probable tokens.

---

8. Prompt Engineering

Topics:

- Writing effective prompts
- Clear instructions
- Role-based prompting
- Providing context
- Output formatting

---

9. Prompting Techniques

Zero-shot Prompting

- Asking models without examples.

One-shot Prompting

- Providing one example.

Few-shot Prompting

- Providing multiple examples.

Chain-of-Thought Prompting

- Breaking complex problems into reasoning steps.

---

10. OpenAI API Basics

Topics:

- API fundamentals
- API keys
- Request/Response flow
- Calling LLM models
- Handling responses

Hands-on Practice

1. Create basic prompts.
2. Compare different prompt styles.
3. Generate structured responses.
4. Create summarization prompts.
5. Build a simple text generation application using API.

Day 17: RAG & GenAI Frameworks

Topics

---

1. Embeddings

- What are embeddings?
- Converting text into numerical vectors.
- Semantic similarity.
- Searching based on meaning.

---

2. Semantic Search

- Keyword search vs semantic search.
- Similarity matching.
- Finding relevant documents.

---

3. Vector Databases

FAISS

- Facebook AI Similarity Search
- Efficient vector storage and retrieval.

ChromaDB

- Open-source vector database.
- Used for storing embeddings.

---

4. Retrieval-Augmented Generation (RAG)

Concept

RAG combines:

- Information Retrieval
- Large Language Models

Workflow:

1. User Query
2. Convert Query into Embedding
3. Search Relevant Documents
4. Retrieve Context
5. Send Context + Query to LLM
6. Generate Answer

---

5. Document Chunking

Topics:

- Why chunking is required.
- Chunk size.
- Chunk overlap.
- Splitting large documents.

---

6. Retriever

Topics:

- Document retrieval
- Similarity search
- Selecting relevant information

---

7. LangChain Basics

Topics:

- LangChain architecture
- Chains
- Prompts
- Memory
- Document loaders
- Retrievers

---

8. LlamaIndex Overview

Topics:

- Data indexing
- Document processing
- Query engines

---

9. Local LLMs (Ollama)

Topics:

- Running local models
- Model management
- Privacy benefits

---

10. Build a Simple RAG Chatbot

Features:

- Upload documents
- Extract text
- Generate embeddings

- Store vectors
- Retrieve information
- Generate answers

Hands-on Practice

1. Generate embeddings from documents.
2. Store embeddings in FAISS/ChromaDB.
3. Perform semantic search.
4. Build a basic RAG pipeline.
5. Create a document question-answering chatbot.

Day 18: Production GenAI

Topics

---

1. AI Agents

Topics:

- What are AI Agents?
- Agent architecture
- Autonomous decision making
- Agent workflows

---

2. Tool Calling / Function Calling

Topics:

- Connecting LLMs with external tools.
- Calling APIs.
- Executing functions dynamically.

Examples:

- Calculator Tool
- Search Tool
- Database Tool

---

3. Fine-tuning vs Pre-training

Pre-training

- Training a model from scratch using large datasets.

Fine-tuning

- Customizing an existing model for specific tasks.

---

4. Hallucinations & Mitigation

Topics:

- Why LLMs hallucinate.
- Reducing incorrect answers.

Methods:

- RAG
- Better prompts
- Validation
- Guardrails

---

5. AI Guardrails & Safety

Topics:

- Responsible AI
- Content filtering
- Input validation
- Output validation

---

6. LLM Evaluation

Metrics:

BLEU

- Measures text similarity.

ROUGE

- Evaluates summaries.

Perplexity

- Measures language prediction ability.

---

7. Memory & Caching

Topics:

- Conversation memory
- Session management
- Response caching
- Performance improvement

---

8. Deployment with FastAPI

Topics:

- Creating API endpoints.
- Connecting LLM applications.
- Handling requests and responses.

---

9. Docker/Git & Git hub/ Jenkins Basics

Topics:

- Containers
- Images
- Dockerfile
- Running GenAI applications
- Git & Git hub – learn all useful comands
- Explore jenkins

---

10. Build an End-to-End GenAI Application

Application Flow:

User → API → AI Model → Processing → Response

Day 19: Advanced GenAI Application Development

Topics

Prompt Engineering Advanced Concepts

- Prompt templates
- Role-based prompts
- Structured outputs
- Prompt optimization

LangChain Application Development

Topics:

- Chains
- Agents
- Tools
- Memory
- Retrieval pipelines

AI Agent Development

Topics:

- Agent planning
- Tool selection
- Multi-step execution
- Agent memory

Hands-on Practice

1. Create reusable prompt templates.
2. Build LangChain workflow.
3. Create AI agent with tools.

4. Connect external APIs.
5. Build automated task execution workflow.

Day 20: GenAI API & Deployment

Topics

FastAPI for GenAI

- Create REST APIs.
- Connect LLM models.
- Handle user requests.
- Return generated responses.

Application Deployment

Topics:

- Environment setup
- Dependency management
- API deployment
- Docker containerization

Hands-on Practice

Build:

- GenAI REST API
- Document processing API
- Chat API

Day 21: Mini Project – Document Q&A Assistant

Project Objective

Build an AI-powered Document Question Answering System.

Features

Document Processing

- Upload PDF
- Extract text
- Process documents

Knowledge Storage

- Create embeddings
- Store vectors in database

Tools:

- FAISS
- ChromaDB

Question Answering

Flow:

User Question → Retrieval → LLM → Answer

User Interface

Build simple interface using:

- Streamlit

Project Architecture

```
User
 |
Streamlit UI
 |
Backend API
 |
Document Processor
 |
Embedding Model
 |
Vector Database
 |
LLM
 |
Response
```

Day 22: End-to-End GenAI Capstone Project

Project: Enterprise Knowledge Assistant

Features

- Document upload
- Semantic search
- RAG pipeline
- Chat interface
- Conversation memory
- API deployment
- Monitoring

Must-Do Hands-on Projects

1. Prompt Engineering using OpenAI API

Skills:

- Prompt creation
- API usage
- Response handling

---

2. Semantic Search using Embeddings

Skills:

- Embedding generation
- Similarity search
- Vector databases

---

3. RAG Chatbot with FAISS/ChromaDB

Skills:

- Document retrieval
- Context generation

- LLM integration

---

4. LangChain-based Q&A Application

Skills:

- Chains
- Retrievers
- Memory

---

5. AI Agent with Tool Calling

Skills:

- Function calling
- Tool integration
- Autonomous workflows

---

6. Deploy a GenAI API using FastAPI

Skills:

- API development
- Deployment
- Docker basics

GenAI 7-Day Completion Outcome

After completing Part II, you will be able to:

Understand Generative AI and LLM fundamentals.

Build effective prompts.

Work with embeddings and vector databases.

Develop RAG-based applications.

Use LangChain and LlamaIndex frameworks.

Build AI agents with tool calling.

Deploy GenAI applications using FastAPI and Docker.

Create real-world GenAI projects.

Final Learning Journey

Python Programming (2 Weeks)

Automation Framework Development

Android Scripting & Testing Automation

Generative AI Application Development

> **"The future belongs to those who continuously learn, experiment, and build."**

---

21 Days Python & Gen AI Practice completed

Part III – Android Automation Scripting

Thundersoft Scripting Questions

ADB & Device Basics

1. Write a script to detect connected Android devices using ADB and validate the device list.
2. Write a script to reboot the device N times and verify boot completion using system properties.
3. Write a script to wait for the device after reboot and validate `sys.boot_completed`.
4. Write a script to build an ADB wrapper using `subprocess.run()` for executing commands.
5. Write a script to handle ADB command failures with exception handling and retry logic.

Wi-Fi

6. Write a script to enable Wi-Fi and validate the state from logs.
7. Write a script to disable Wi-Fi and validate the state from logs.
8. Write a script to connect to Wi-Fi and verify IP address assignment.
9. Write a script to reboot the device and verify Wi-Fi reconnects automatically.
10. Write a script to toggle Wi-Fi N times and generate a pass/fail report.

Camera

11. Write a script to launch the camera application and validate successful launch.
12. Write a script to take a photo and validate image file creation.
13. Write a script to capture a screenshot while the camera is open and validate the file.
14. Write a script to launch the camera N times and detect failures from logs.
15. Write a script to monitor camera memory usage and detect leaks.

Bluetooth

16. Write a script to enable Bluetooth and validate the adapter state.
17. Write a script to pair a Bluetooth device and verify connection.
18. Write a script to reboot the device and verify Bluetooth reconnects automatically.
19. Write a script to toggle Bluetooth repeatedly and detect failures.
20. Write a script to collect Bluetooth logs during failures and save them to a file.

Log Collection & Analysis

21. Write a script to capture logcat and save it to a file.
22. Write a script to capture logs during reboot and validate continuity.
23. Write a script to search logs for CRASH events and report them.
24. Write a script to search logs for FATAL EXCEPTION and extract stack traces.
25. Write a script to search logs for ANR events and report affected apps.
26. Write a script to generate a crash summary report from logs.
27. Write a script to count errors and warnings from logs and summarize them.

APK Automation

28. Write a script to install an APK and validate installation success.
29. Write a script to uninstall an APK and validate removal.
30. Write a script to launch an application and verify running state.
31. Write a script to force-stop an application and validate termination.
32. Write a script to install/uninstall APKs repeatedly and generate a report.

Memory & Performance

33. Write a script to capture RAM usage and monitor memory spikes.
34. Write a script to capture CPU usage and monitor utilization.
35. Write a script to detect memory leaks after repeated camera launches.
36. Write a script to generate CPU and memory utilization reports.
37. Write a script to monitor memory while an app is running continuously.

Battery

38. Write a script to read battery percentage and report the value.
39. Write a script to verify charging state and report status.
40. Write a script to monitor battery level continuously and detect drain.
41. Write a script to monitor battery temperature and report anomalies.
42. Write a script to generate a battery report in CSV/HTML format.

Threading

43. Write a script to capture logcat while rebooting the device in parallel.
44. Write a script to monitor battery while recording video simultaneously.
45. Write a script to run Wi-Fi validation and log collection in parallel.
46. Write a script to execute tests on multiple Android devices simultaneously.

Reports

47. Write a script to generate a CSV report summarizing test results.
48. Write a script to generate an HTML report for test execution.
49. Write a script to create a pass/fail dashboard for test cases.

50. Write a script to generate an execution summary report.

GNSS (GPS)

51. Write a script to verify GPS enable/disable and validate logs.
52. Write a script to get current latitude/longitude and compare with expected.
53. Write a script to measure GPS TTFF (Time To First Fix).
54. Write a script to verify GPS state after reboot.
55. Write a script to collect GNSS logs and verify satellite lock.

Display

56. Write a script to change brightness and validate the property update.
57. Write a script to turn display ON/OFF and validate state.
58. Write a script to verify display settings persist after reboot.
59. Write a script to capture screenshots periodically and validate files.
60. Write a script to verify screen timeout settings and behavior.

Telephony / WWAN

61. Write a script to verify SIM detection and validate logs.
62. Write a script to verify LTE/5G registration and connectivity.
63. Write a script to toggle airplane mode and validate state.
64. Write a script to verify mobile data connectivity and ping gateway.
65. Write a script to monitor signal strength and report fluctuations.

Audio

66. Write a script to increase/decrease volume and validate changes.
67. Write a script to mute/unmute the device and validate state.
68. Write a script to play audio and validate playback.
69. Write a script to verify audio playback after reboot.
70. Write a script to monitor audio service logs and detect errors.

Video

71. Write a script to play video and validate playback.
72. Write a script to record video and validate file creation.
73. Write a script to verify video playback after reboot.
74. Write a script to monitor video playback for crashes.
75. Write a script to collect media logs during playback.

Sensors

76. Write a script to verify accelerometer availability and validate values.
77. Write a script to verify gyroscope availability and validate values.
78. Write a script to verify proximity sensor and validate events.
79. Write a script to validate sensor service is running.
80. Write a script to compare sensor list before and after reboot.

Android Framework

81. Write a script to verify Activity Manager service and validate app launch.

82. Write a script to verify Package Manager service and validate APK install.
83. Write a script to verify Wi-Fi service and validate toggling.
84. Write a script to verify Sensor service and validate response.
85. Write a script to generate an Android service report.

Storage

86. Write a script to push a file to the device and validate transfer.
87. Write a script to pull a file from the device and validate integrity.
88. Write a script to monitor free storage and report usage.
89. Write a script to create/delete files repeatedly and validate stability.
90. Write a script to verify storage mounts after reboot.

Libraries & Vendor Files

91. Write a script to verify `.so` library existence.
92. Write a script to verify vendor library loaded successfully.
93. Write a script to compare library versions and report mismatches.
94. Write a script to verify vendor configuration files exist and are applied.
95. Write a script to generate a loaded library report.

Top 20 Scripting Questions to Prepare Before Any Interview

1. Write a script to reboot the device N times and verify boot completion using system properties.
2. Write a script to build an ADB wrapper using `subprocess.run()` for executing commands.
3. Write a script to enable and disable Wi-Fi, then validate the state from logs.
4. Write a script to reboot the device and verify Wi-Fi reconnects automatically.
5. Write a script to launch the camera application and validate successful launch.
6. Write a script to capture a screenshot while the camera is open and validate the file creation.
7. Write a script to install and uninstall APKs repeatedly, validating success and generating a report.
8. Write a script to capture logcat output and save it to a file for analysis.
9. Write a script to search logcat for crash events and generate a crash detection summary.
10. Write a script to detect ANR (Application Not Responding) events from logs and report them.
11. Write a script to monitor battery percentage and charging state continuously.
12. Write a script to monitor CPU usage and generate utilization reports.
13. Write a script to monitor RAM usage and detect abnormal memory consumption.
14. Write a script to detect memory leaks after repeated camera launches using dumpsys data.
15. Write a script to enable and disable Bluetooth repeatedly and validate connection state.
16. Write a script to perform basic GPS validation (enable/disable, location fix, logs).
17. Write a script to run Wi-Fi validation and log collection in parallel using threading.
18. Write a script to generate a CSV report summarizing test results.
19. Write a script to handle ADB command failures with exception handling and retry logic.

MORE QUESTIONS TO PRACTICE AND IMPROVE LOGIC BUILDING:

Python Practical Logic & Automation Task List

Regex & Validation (9 tasks)

1. Email validation
2. Indian mobile number validation
3. Extract numbers from paragraph
4. Custom strip() using regex
5. Mask Aadhaar/account/mobile numbers
6. Validate date format DD/MM/YYYY
7. Validate vehicle registration numbers
8. Extract URLs from file
9. Validate IPv4 address

File Handling & Automation (21 tasks)

10. Find duplicate words in text file
11. Search word in multiple .txt files
12. Rename multiple files automatically
13. Organize files by extension
14. Count word frequency in file
15. Extract email IDs from file
16. Replace bad words dynamically
17. Generate random quiz file
18. Read CSV & generate summary
19. Compare two text files
20. Remove extra spaces/newlines
21. Regex search in files
22. Automatic backup copies
23. Detect duplicate files
24. Extract hashtags & mentions
25. Log analyzer
26. Calculate file sizes recursively
27. Merge multiple text files
28. Split large text files
29. Mini contact-book system

Security & String Manipulation (15 tasks)

30.Password generator
31.Password strength validator
32.String compression
33.Reverse alphabets only
34.First non-repeated character
35.Anagram check
36.Remove duplicate characters
37.Longest & shortest words
38.Count vowels/consonants/digits/specials
39.CamelCase ↔ snake_case
40.Username suggestions
41.Palindromic words
42.Remove consecutive duplicates
43.Caesar Cipher
44.Word grouping (anagrams)

Math & Algorithms (21 tasks)

45.Armstrong, Perfect, Strong numbers
46.Fibonacci (recursion & loop)
47.Flatten nested lists
48.Custom len()
49.Common elements in lists
50.Matrix addition & multiplication
51.Sort list of dictionaries
52.Second largest number
53.Rotate string by N
54.Balanced brackets
55.Missing numbers in sequence
56.Move zeros to end
57.Longest substring without repeats
58.Patterns (pyramid, diamond, Pascal)
59.Pair sum equals target
60.Factorial
61.Prime number
62.Find max number
63.Second largest & second minimum
64.LCM & HCF
65.Most repeated numbers

Classic Logic Problems (19 tasks)

66.Palindrome (integer)
67.Palindrome (string)
68.Reverse string (3 ways)
69.Reverse words in sentence
70.Letter frequency
71.Word frequency
72.Nested dictionary operations
73.Remove duplicates
74.Most frequent 2 words
75.Sort mixed list
76.Sort dictionary by keys
77.Sort dictionary by values
78.Reverse alphabets only (again)
79.Reverse words in string
80.Palindrome check (extra practice)
81.Count word frequency (extra practice)
82.Count letter frequency (extra practice)
83.Find most frequent numbers